Object Oriented Programming

Dr. Imran Ashraf
Office: 02
imran.ashraf[at]hitecuni[dot]edu[dot]pk

Last Week

- Polymorphism
  - Virtual Functions
  - Pure Virtual Functions
  - Abstract Base Classes

This Week

- Streams

Streams

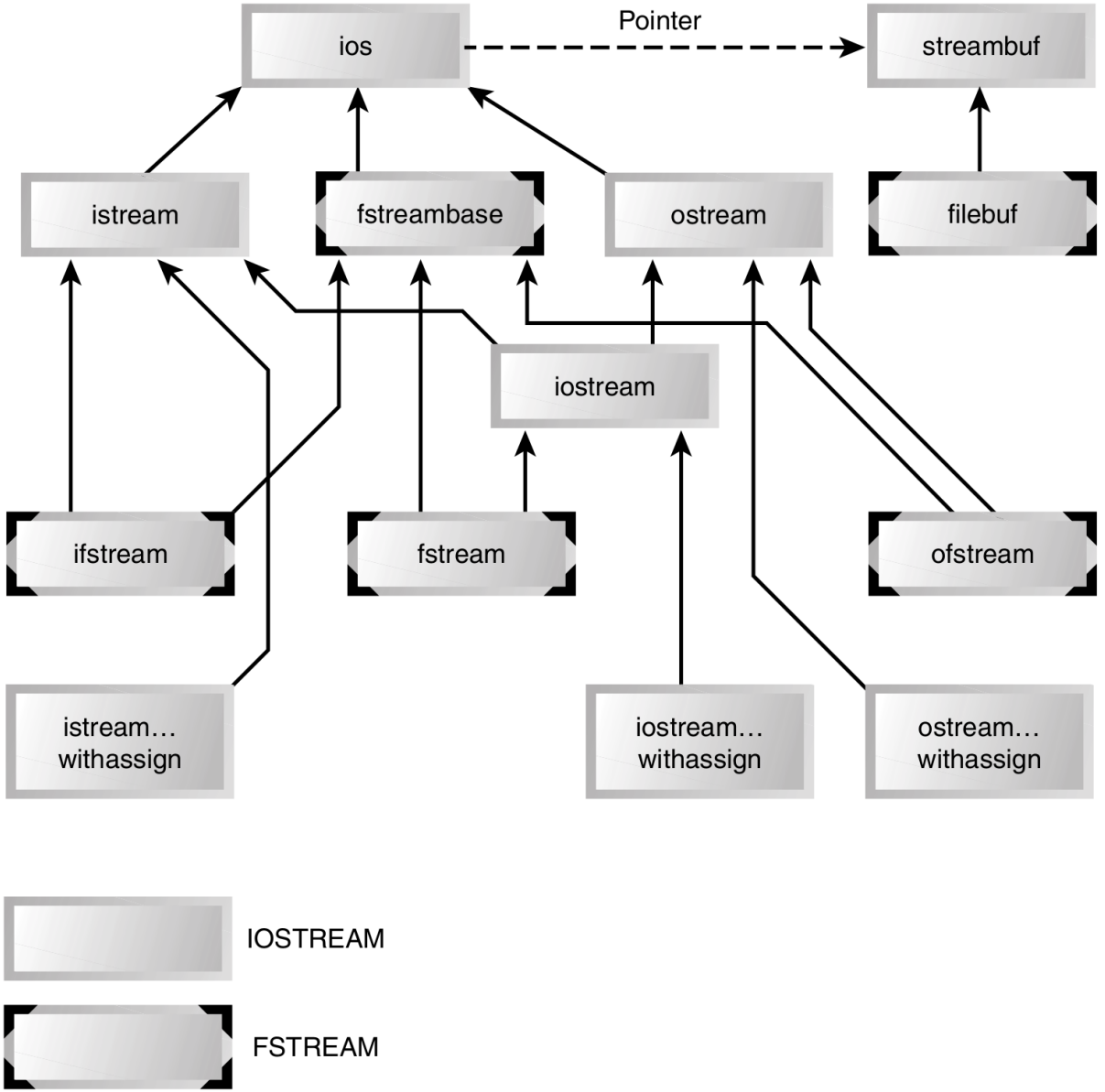
- We have already been using streams
  - Cin
  - Cout
- Stream is a general name given to flow of data
- Different streams represnet different kind of data flow
  - ifstream
  - ofstream

Traditional C

- printf/scanf
- fprintf/fscanf

- Simplicity (no formatting characters)
- Overload insertion? / extraction? operators for your own data types

Class hierarchy



Formatting Flags

Flag	*Meaning*
`skipws`	Skip (ignore) whitespace on input
`left`	Left-adjust output [`12.34 `]
`right`	Right-adjust output [`12.34`]
`internal`	Use padding between sign or base indicator and number [`+ 12.34`]
`dec`	Convert to decimal
`oct`	Convert to octal
`hex`	Convert to hexadecimal
`boolalpha`	Convert bool to “true” or “false” strings
`showbase`	Use base indicator on output (0 for octal, 0x for hex)
`showpoint`	Show decimal point on output
`uppercase`	Use uppercase `X`, `E`, and hex output letters (`ABCDEF`)—the default is lowercase
`showpos`	Display `+` before positive integers
`scientific`	Use exponential format on floating-point output [`9.1234E2`]
`fixed`	Use fixed format on floating-point output [`912.34`]
`unitbuf`	Flush all streams after insertion
`stdio`	Flush `stdout`, `stderror` after insertion

Examples

- left justify output text
  - cout.setf(ios::left);
  - cout >> “This text is left-justified”;
- return to default (right justified)
  - cout.unsetf(ios::left);
- Change base
  - Cout << hex;

Manipulators: no args

Manipulator	*Purpose*
`ws`	Turn on whitespace skipping on input
`dec`	Convert to decimal
`oct`	Convert to octal
`hex`	Convert to hexadecimal
`endl`	Insert newline and flush the output stream
`ends`	Insert null character to terminate an output string
`flush`	Flush the output stream
`lock`	Lock file handle
`unlock`	Unlock file handle

You insert these manipulators directly into the stream. For example, to output `var` in hexadecimal format, you can say

```
cout << hex << var;
```

Manipulators: with args

Manipulator	*Argument*	*Purpose*
`setw()`	field width (`int`)	Set field width for output
`setfill()`	fill character (`int`)	Set fill character for output (default is a space)
`setprecision()`	precision (`int`)	Set precision (number of digits displayed)
`setiosflags()`	formatting flags (`long`)	Set specified flags
`resetiosflags()`	formatting flags (`long`)	Clear specified flags

Examples

```
float f = 3.14159;
cout << setprecision(9) << f << endl; // prints "3.14159"
cout << fixed << setprecision(9) << f << endl; // prints "3.141590000"
```

```
float f = 3.14159;
cout.precision(9);
cout << f << endl; // prints "3.14159"
cout.setf(ios::fixed, ios::floatfield); //when a float is provided, the precision is fixed.
cout << f << endl; // prints "3.141590000"
```

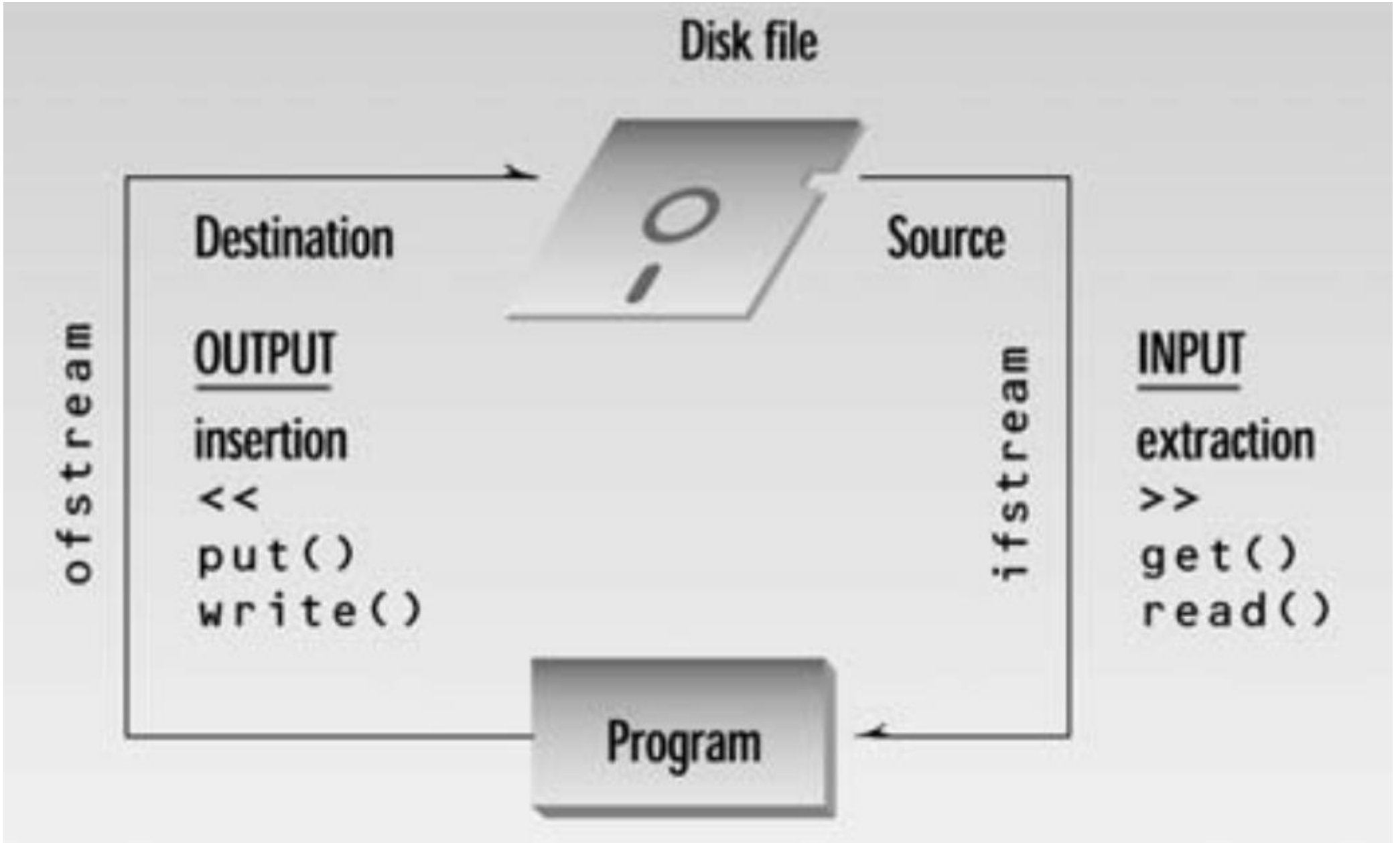
Useful Functions for iostreams

Function	*Purpose*
`ch = fill();`	Return the fill character (fills unused part of field; default is space)
`fill(ch);`	Set the fill character
`p = precision();`	Get the precision (number of digits displayed for floating-point)
`precision(p);`	Set the precision
`w = width();`	Get the current field width (in characters)
`width(w);`	Set the current field width
`setf(flags);`	Set specified formatting flags (for example, `ios::left`)
`unsetf(flags);`	Unset specified formatting flags
`setf(flags, field);`	First clear field, then set flags

Examples

- cout.width(14);
- cout.fill(‘*’);
- cout.setf(ios::left);
- cout.unsetf(ios::left);

File I/O


Disk file
Destination
Source
OUTPUT
insertion
<<
put()
write()
INPUT
extraction
>>
get()
read()
ofstream
ifstream
Program

Pre-defined Streams

- cin
- cout
- cerr (critical errors)
- clog (non-critical errors)

Writing File

```
#include <fstream>
...
int main()
{
    string str(“This goes to file”);
    ofstream outfile(“fdata.txt”);
    outfile << str << endl;
    return 0;
}
```

Reading File

```
#include <fstream>
…

int main()
{
    string str;
    ifstream infile(“fdata.txt”);
    infile >> mystr;
    return 0;
}
```

Summary

- C++ streams provide easy to use way to deal with file I/O
- Various manipulators